LAPORAN PRAKTIKUM

ALGORITMA DAN STRUKTUR DATA

Disusun Oleh:

Nama : **M. SYAUQI AUFA IZZAZULLAH**

NIM : **2025573010075**

Kelas : **1D – TI**

Prodi : **Teknologi Informasi Dan Komputer**

Pembimbing : **M. REZA ZULMAN, S.ST., M.SC**

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE 2026

Kata pengantar

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban penulis setelah melaksanakan kegiatan praktikum.

Penulis menyadari bahwa dalam penyusunan laporan praktikum ini masih terdapat berbagai kekurangan. Namun berkat bantuan, bimbingan, serta dukungan dari berbagai pihak, laporan ini dapat diselesaikan. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan terima kasih kepada dosen pengampu mata praktikum serta semua pihak yang telah membantu dalam pelaksanaan praktikum hingga penyusunan laporan ini.

Penulis berharap laporan praktikum ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca, khususnya bagi penulis sendiri. Penulis juga mengharapkan kritik dan saran yang membangun demi perbaikan laporan ini di masa yang akan datang.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi semua pihak yang membacanya.

Buketrata,…… 2026

M. Syauqi Aufa Izzazullah

DAFTAR ISI

DAFTAR ISI .....
PENDAHULUAN.....
1.1 Latar Belakang.....
1.2 Rumusan Masalah.....
1.3 Tujuan Praktikum.....

BAB II DASAR TEORI.....
2.1 Mengapa Efisiensi Algoritma Penting?.....
2.2 Notasi Big O.....
2.3 Space Complexity.....
2.4 Mengenali Pola Kompleksitas.....
2.5 Operator Java Script.....
2.6 Kondisional.....

BAB III PEMBAHASAN.....
3.1 Persiapan Project.....
3.2 Mengukur Waktu Eksekusi Empiris.....
3.3 Contoh Tiap Kelas Big O.....
3.4 Space Complexity.....
3.5 Refaktor Kode yang Lambat.....

BAB IV Tugas Praktikum.....
4.1 Tugas 1: Analisis dan Refactor.....
4.2 Tugas 2: Visualisasi Pertembuhan BIG O.....

BAB IV PENUTUP.....
4.1 Kesimpulan.....

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, menulis program yang benar saja tidak cukup. Sebuah algoritma juga harus efisien agar mampu menangani data dalam jumlah besar dengan waktu eksekusi dan penggunaan memori yang optimal. Pada skala kecil, perbedaan performa algoritma mungkin tidak terlalu terlihat, namun ketika ukuran data bertambah besar, algoritma yang tidak efisien dapat menyebabkan program berjalan sangat lambat bahkan tidak praktis digunakan. Oleh karena itu, pemahaman tentang analisis kompleksitas algoritma menjadi fondasi penting dalam mempelajari struktur data dan algoritma.

Analisis kompleksitas algoritma mempelajari bagaimana kebutuhan waktu (time complexity) dan memori (space complexity) suatu program bertumbuh seiring bertambahnya ukuran input. Konsep ini umumnya direpresentasikan menggunakan Big O Notation, seperti O(1), O(log n), O(n), O(n log n), hingga $O(n^2)$, yang digunakan untuk mengukur laju pertumbuhan performa algoritma. Dengan memahami Big O, mahasiswa dapat mengevaluasi apakah suatu solusi cukup efektif, membandingkan beberapa pendekatan, serta melakukan refactor terhadap kode yang lambat menjadi lebih optimal.

Pada praktikum Modul 5 ini, mahasiswa mempelajari konsep dasar analisis kompleksitas melalui implementasi berbagai algoritma menggunakan JavaScript dan Node.js. Praktikum mencakup pengukuran performa empiris menggunakan `Date.now()`, pengenalan berbagai kelas kompleksitas, analisis penggunaan memori, serta optimasi kode menggunakan struktur data seperti Set dan Map. Melalui kegiatan ini, mahasiswa diharapkan mampu mengenali pola kode yang kurang efisien, memahami trade-off antara waktu dan ruang, serta menerapkan solusi algoritmik yang lebih baik dalam pemecahan masalah komputasi.

1.2 Rumusan Masalah

1. Mengapa analisis efisiensi algoritma penting dalam pengembangan program, terutama pada data berukuran besar?

2. Bagaimana cara memahami dan menerapkan konsep Big O Notation dalam menganalisis performa algoritma?

3. Bagaimana membedakan berbagai kelas kompleksitas seperti O(1), O(log n), O(n), O(n log n), $O(n^2)$, dan $O(2^n)$ berdasarkan pola kode yang digunakan?

4. Bagaimana cara mengukur dan membandingkan waktu eksekusi beberapa algoritma secara empiris menggunakan JavaScript?

5. Bagaimana konsep Space Complexity digunakan untuk menganalisis penggunaan memori pada suatu algoritma?

6. Bagaimana melakukan refactor terhadap algoritma yang memiliki kompleksitas buruk agar menjadi lebih efisien dengan memanfaatkan struktur data yang tepat seperti Set, Map, atau Object?

7. Bagaimana hubungan antara teori kompleksitas algoritma dengan implementasi praktis dalam penyelesaian masalah pemrograman sehari-hari?

1.3 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami mengapa analisis efisiensi algoritma itu penting.
2. Menjelaskan konsep *Time Complexity* dan *Space Complexity*.
3. Membaca dan menulis notasi Big O seperti O(1), O(log n), O(n), O(n log n), dan $O(n^2)$.
4. Menganalisis kompleksitas pada loop tunggal maupun nested loop.
5. Membandingkan efisiensi dua algoritma secara empiris menggunakan fungsi `Date.now()`.
6. Mengenali pola kode yang memiliki kompleksitas buruk serta mengetahui cara memperbaikinya.

BAB ll

DASAR TEORI

2.1 Mengapa Efisiensi Algoritma Penting?

Bayangkan kamu punya daftar 10 nama. Mencarinya satu per satu paling lama butuh 10 langkah — tidak masalah. Sekarang bayangkan aplikasi seperti Google dengan miliaran data. Algoritma yang 'cukup baik' untuk 10 data bisa membutuhkan jutaan tahun untuk miliaran data jika algoritmanya buruk. Itulah mengapa kita perlu cara formal untuk mengukur efisiensi: Big O Notation. Big O menjawab pertanyaan: 'Bagaimana waktu/memori yang dibutuhkan algoritma bertumbuh seiring bertambahnya data input?' Perlu diingat: Big O mengukur laju pertumbuhan (growth rate), bukan waktu absolut dalam detik, karena waktu absolut bergantung pada hardware.

2.2 Notasi Big O

Berikut kelas-kelas Big O dari yang paling cepat ke paling lambat:

▪ O(1) — Konstan. Waktu eksekusi tidak bergantung pada ukuran input sama sekali.

▪ Contoh: akses elemen array dengan indeks, operasi aritmatika, akses property object.

▪ O(log n) — Logaritmik. Setiap langkah memotong masalah menjadi separuh.

▪ Contoh: Binary Search — mencari di 1 juta data hanya butuh ~20 langkah.

▪ O(n) — Linear. Waktu tumbuh sebanding lurus dengan ukuran input.

▪ Contoh: loop sekali melewati array, Linear Search.

▪ O(n log n) — Linearitmik. Sedikit lebih buruk dari linear.

▪ Contoh: Merge Sort, Quick Sort — algoritma sorting terbaik yang bisa dicapai.

▪ $O(n^2)$ — Kuadratik. Nested loop — sangat lambat untuk data besar.

▪ Contoh: Bubble Sort, Selection Sort, nested loop dua tingkat.

▪ $O(2^n)$ — Eksponensial. Sangat buruk — hanya bisa digunakan untuk n sangat kecil.

▪ Contoh: Fibonacci rekursif naif, subset generation.

Aturan penyederhanaan Big O:

1. Buang konstanta: $O(2n) \rightarrow O(n)$, $O(100) \rightarrow O(1)$

2. Buang suku kecil: $O(n^2 + n) \rightarrow O(n^2)$, $O(n + \log n) \rightarrow O(n)$

3. Perhatikan suku yang paling dominan seiring $n \rightarrow \infty$

2.3 Space Complexity

Selain waktu, kita juga harus mempertimbangkan memori yang digunakan algoritma. Inilah yang disebut Space Complexity.

Space complexity mengukur memori tambahan (auxiliary space) yang dibutuhkan algoritma seiring bertambahnya input — tidak termasuk input itu sendiri.

Contoh perbandingan:

▪ Algoritma yang hanya menggunakan beberapa variabel → O(1) space

▪ Algoritma yang membuat array baru sebesar n → O(n) space

▪ Rekursi sedalam n tingkat → O(n) space (call stack)

Trade-off yang umum: Banyak algoritma bisa dibuat lebih cepat dengan mengorbankan lebih banyak memori, atau sebaliknya. Ini disebut time-space trade-off.

Contoh: memoization membuat Fibonacci dari $O(2^n)$ time → O(n) time, tapi memerlukan O(n) space tambahan.

2.4 Mengenali Pola Kompleksitas

Tabel Referensi

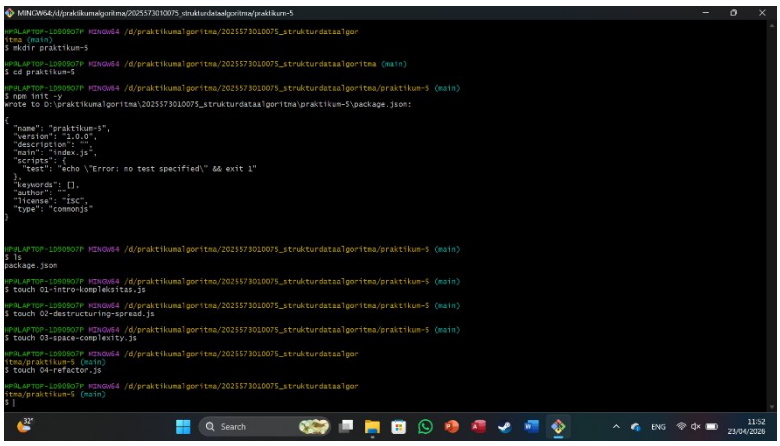
Pola Kode	Kompleksitas	Contoh Nyata
Akses array/object langsung	O(1)	arr[0], obj.key, Map.get()
Satu loop melewati n elemen	O(n)	forEach, for loop linear
Loop i*=2 atau i/=2	O(log n)	Binary search, halving loop
Dua loop berurutan (bukan nested)	O(n)	2n → O(n) setelah buang konstanta
Dua loop bersarang (nested)	$O(n^2)$	Bubble sort, cari pasangan naif
Tiga loop bersarang	$O(n^3)$	Jarang tapi ada di algoritma matrix
Rekursi bercabang dua	$O(2^n)$	Fibonacci naif, power set
Rekursi + sorting	O(n log n)	Merge sort, quick sort rata-rata

BAB III

PEMBAHASAN

3.1 Persiapan Project

1. **Buat folder baru praktikum-5/ didalam repository Github dan inisialisasi npm project.**
2. **Buka Gitbash, navigasikan ke root folder repository kamu.**
   [cd nama-repository-kamu]
3. **Buat folder praktikum-5/ dan masuk kedalamnya.**
   [mkdir praktikum-5
   cd praktikum-5]
4. **Inisialisasi npm project.**
   [npm init -y]



Struktur Folder Yang diharapkan di akhir praktikum :

Struktur folder yang diharapkan di akhir praktikum:

```
nama-repository/
└── praktikum-4/
    ├── package.json
    ├── 01-intro-kompleksitas.js
    ├── 02-kelas-big-o.js
    ├── 03-space-complexity.js
    ├── 04-refactor.js
    ├── latihan-1.js
    └── tugas/
        ├── tugas-1.js
        └── tugas-2.js
```

3.2 Mengukur Waktu Eksekusi Empiris

1. Di dalam folder praktikum-5/, buat file baru bernama 01-intro-kompleksitas.js

 # Pastikan kamu berada di dalam folder praktikum-5

touch 01-intro-kompleksitas.js

2. Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

JS 01-intro-kompleksitas.js U

praktikum-5 > JS 01-intro-kompleksitas.js > ...

```javascript
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}

const N = 100_000;

function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++) total += i;
    return total;
}

function jumlahkanRumus(n) {
    return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
    const pasangan = [];
    for ( let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
        if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
    return pasangan;
}


const data = Array.from({length: 5000}, (_, i) => i);
console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('o(1) jumlahkanRumus ', () => jumlahkanRumus(N));
ukurWaktu('o(n) jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('o((n²) cariPasangan ', () => cariPasangan(data));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-5
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
o(1) jumlahkanRumus : 0 ms
o(n) jumlahkanLinear : 1 ms
o((n²) cariPasangan : 14 ms

Hasil sama? true
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5>
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.
node 01-intro-kompleksitas.js

3.3 Contoh Tiap Kelas Big O

1. Di dalam folder praktikum-5/, buat file baru bernama 02-destructuring-spread.js
 # Pastikan kamu berada di dalam folder praktikum-5
touch 02-destructuring-spread.js
2. Buka file 02-destructuring-spread.js di VS Code, lalu ketikkan kode berikut.

```
JS 02-destructuring-spread.js U ×
praktikum-5 > JS 02-destructuring-spread.js > ...
1  function ambilPertama(arr)  { return arr[0]; }
2  function tambahkanItem(arr, item) { arr.push(item); }
3  function isGenap(n) { return n % 2 === 0; }
4
5  function binarySearch(arr, target) {
6      let kiri = 0, kanan = arr.length - 1;
7      let langkah = 0;
8      while (kiri <= kanan) {
9          langkah++;
10         const tengah = Math.floor((kiri + kanan) / 2);
11         if (arr[tengah] === target) {
12             console.log(`  Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
13             return tengah;
14         }
15         if (arr[tengah] < target) kiri = tengah + 1;
16         else kanan = tengah - 1;
17     }
18     return -1;
19 }
20
21 function cariMax(arr) {
22     let maks = arr[0];
23     for (const x of arr) if (x > maks) maks = x;
24     return maks;
25 }
26
27 function bubbleSortNaif(arr) {
28     const a = [...arr];
29     for (let i = 0; i < a.length; i++)
30         for (let j = 0; j < a.length - i - 1; j++)
31             if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
32     return a;
33 }
34
35 function fibRekursif(n) {
36     if (n <= 1) return n;
37     return fibRekursif(n-1) + fibRekursif(n-2);
38 }
```

```
39
40 console.log('=== O(1) — selalu cepat ===');
41 console.log(ambilPertama([10,20,30,40,50]));
42 console.log(isGenap(42));
43
44 console.log('\n=== O(log n) — Binary Search ===');
45 const sorted = Array.from({length:1_000_000},(_,i)=>i);
46 binarySearch(sorted, 731_452);
47
48 console.log('\n=== O(n) — Linear Search ===');
49 console.log('Max dari 1000 elemen:',
50 cariMax(Array.from({length:1000},()=>Math.random()*1000|0)));
51
52 console.log('\n=== O(n²) — Bubble Sort (kecil saja) ===');
53 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
54
55 console.log('\n=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===');
56 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
57 console.log('');
58 console.log('\nWaktu fib(35) O(2ⁿ):');
59 let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');
60
61 console.log('Waktu fib(35) O(n) memoization:');
62 const memo = {};
63 function fibMemo(n) {
64     if (n <= 1) return n;
65     if (memo[n]) return memo[n];
66     return memo[n] = fibMemo(n-1) + fibMemo(n-2);
67     }
68
69 t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
70
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> node 02-destructuring-spread.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
  Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 998

=== O(n²) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
82 ms
Waktu fib(35) O(n) memoization:
0 ms
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5>
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.
node 02-destructuring-spread.js

➢ Latihan

Latihan 1: Identifikasi Kompleksitas

1. Buat file baru `latihan-1.js`.
2. Untuk setiap potongan kode di bawah ini, tulis dalam komentar Big O-nya dan alasannya:
3. Fungsi A: `function fn(n) { return n * 2; }` → Big O: ?
4. Fungsi B: `for (let i=0; i<n; i++) for (let j=0; j<n; j++) console.log(i,j);` → Big O: ?
5. Fungsi C: `for (let i=1; i<n; i*=2) console.log(i);` → Big O: ?
6. Fungsi D: `arr.forEach(x => arr.forEach(y => arr.forEach(z => console.log(x,y,z))));` → Big O: ?
7. Implementasikan dan jalankan setiap fungsi, ukur waktunya dengan `Date.now()` untuk n=1000.
8. Buat function `hitungKompleksitas(n, fn)` yang menjalankan fn(n) dan mencetak waktu eksekusi.

latihan-1.js U

praktikum-5 > latihan-1.js > ...

```javascript
// Latihan 1: Identifikasi Kompleksitas

function f(n){
  return n * 2;
}

function fungsiB(n){
  let count = 0;
  for (let i=0; i<n; i++){
    for (let j=0; j<n; j++){
      count++;
    }
  }
  return count;
}

function fungsiC(n){
  let count = 0;
  for (let i=1; i<n; i*=2){
    count++;
  }
  return count;
}

function fungsiD(arr){
  let count = 0;
  arr.forEach(x => arr.forEach(y => count++));
  return count;
}

function hitungKompleksitas(input, fn){
  const mulai = Date.now();
  const hasil = fn(input);
  const selesai = Date.now();
  console.log("Waktu:", selesai - mulai, "ms | hasil:", hasil);
}

const n = 1000;
const data = Array.from({length: n}, (_, i) => i);

hitungKompleksitas(n, f);
hitungKompleksitas(n, fungsiB);
hitungKompleksitas(n, fungsiC);
hitungKompleksitas(data, fungsiD);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
0 ms
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> node latihan-1.js
Waktu: 0 ms | hasil: 2000
Waktu: 4 ms | hasil: 1000000
Waktu: 0 ms | hasil: 10
Waktu: 9 ms | hasil: 1000000
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5>
```

3.4 Space Complexity

1. Di dalam folder praktikum-5/, buat file baru bernama 03-space-complexity.js

 # Pastikan kamu berada di dalam folder praktikum-5

touch 03-space-complexity.js

2. Buka file 03-space-complexity.js. di VS Code, lalu ketikkan kode berikut.

```
JS 03-space-complexity.js U ×
praktikum-5 > JS 03-space-complexity.js > jumlahArray
1  function jumlahArray(arr) {
2      let total = 0;
3      for (const x of arr) total += x;
4      return total;
5  }
6
7  function duplikasiArray(arr) {
8      const baru = [];
9      for (const x of arr) baru.push(x * 2);
10     return baru;
11 }
12
13 function faktorialRekursif(n) {
14     if (n <= 1) return 1;
15     return n * faktorialRekursif(n - 1);
16 }
17
18 function faktorialIteratif(n) {
19     let hasil = 1;
20     for (let i = 2; i <= n; i++) hasil *= i;
21     return hasil;
22 }
23
24 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
25
26 console.log('Jumlah array   :', jumlahArray(arr));
27 console.log('Duplikasi array:', duplikasiArray(arr));
28 console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
29 console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
30
31 function hitungUnik(arr) {
32     const seen = new Set();
33     for (const x of arr) seen.add(x);
34     return seen.size;
35 }
36
37 const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
38 console.log('Elemen unik:', hitungUnik(dataAcak));
39
```

```
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> node 03-space-complexity.js
Jumlah array   : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.

node 03-space-complexity.js

➢ Latihan

Latihan 3: Class BankAccount

1. Tambahkan kode di bawah file `04-space-complexity.js`.
2. Buat dua fungsi untuk menemukan dua angka dalam array yang jumlahnya = target:
   - `cariPasanganLambat(arr, target)` — gunakan nested loop. Tulis Big O time & space-nya dalam komentar.
   - `cariPasanganCepat(arr, target)` — gunakan `Set` untuk menyimpan angka yang sudah dilihat. Cek apakah `target - elemen` ada di Set. Tulis Big O-nya.
3. Uji kedua fungsi dengan `[2, 7, 11, 15]` dan target `9`. Keduanya harus mengembalikan `[2, 7]`.
4. Ukur waktu keduanya dengan array berisi 50.000 angka acak dan target yang tidak ada di array.
5. Diskusi: Fungsi mana yang lebih baik? Apa trade-off-nya?

```javascript
// Latihan 2: ClassBankAccount

function cariPasanganLambat(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }
        }
    }
    return null;
}

function cariPasanganCepat(arr, target) {
    const seen = new Set();
    for (const x of arr) {
        const pasangan = target - x;
        if (seen.has(pasangan)) {
            return [pasangan, x];
        }

        seen.add(x);
    }

    return null;
}

const dataUji = [2, 7, 11, 15];
const target = 9;

console.log("=== Uji Fungsi ===");
console.log("Lambat:", cariPasanganLambat(dataUji, target));
console.log("Cepat :", cariPasanganCepat(dataUji, target));

function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}

const dataBesar = Array.from({ length: 50000 }, () => Math.floor(Math.random() * 100000));
const targetBesar = -1;

ukurWaktu("Lambat O(n²)", () => cariPasanganLambat(dataBesar, targetBesar));
ukurWaktu("Cepat  O(n) ", () => cariPasanganCepat(dataBesar, targetBesar));
```

```
=== Uji Fungsi ===
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]
Lambat O(n²): 1280 ms
Cepat  O(n) : 10 ms
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5>
```

3.5 Refaktor Kode yang Lambat

1. Di dalam folder praktikum-5/, buat file baru bernama 04-refactor.js

 # Pastikan kamu berada di dalam folder praktikum-5

touch 04-refactor.js

2. Buka file 04-refactor.js. di VS Code, lalu ketikkan kode berikut.

```
JS 04-refactor.js U ×
praktikum-5 > JS 04-refactor.js > adaDuplikatLambat
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
    for (let j = i+1; j < arr.length; j++)
    if (arr[i] === arr[j]) return true;
    return false;
}

function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
    }
    return false;
}

function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}

function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}

const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT  O(n) :', Date.now()-t, 'ms');

const kata = ['a','b','a','c','b','a','d'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
```

```
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT  O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5>
```

BAB IV

TUGAS PRAKTIKUM

Tugas 1 : Analisis Dan Refactor

Tugas 1: Analisis dan Refactor

1. Buat file tugas/tugas-1.js.
2. Terdapat tiga fungsi berikut yang perlu kamu analisis dan refactor:
3. • Fungsi A: menerima dua array, kembalikan array berisi elemen yang ada di keduanya (intersection). Implementasikan versi $O(n^2)$ dan $O(n)$ menggunakan Set.
4. • Fungsi B: diberikan array string, kembalikan kelompok anagram. Contoh: ['eat','tea','tan','ate','nat','bat'] → [['eat','tea','ate'],['tan','nat'],['bat']]. Hint: sortir setiap kata sebagai key.
5. • Fungsi C: diberikan array angka positif, kembalikan true jika ada dua elemen yang jumlahnya sama dengan kuadrat elemen ketiga. Buat versi $O(n^3)$ dan $O(n \log n)$.
6. Untuk setiap fungsi: tulis Big O time & space dalam komentar, ukur waktu eksekusi keduanya dengan data besar.
7. Simpan dan jalankan: node tugas/tugas-1.js.

tugas-1.js U

praktikum-5 > tugas > tugas-1.js > ...

```javascript
// TUGAS 1: Analisis dan Refactor

function ukurWaktu(label, fn) {
    const mulai = Date.now();
    const hasil = fn();
    const selesai = Date.now();

    console.log(`${label}: ${selesai - mulai} ms`);
    return hasil;
}

// FUNGSI A: INTERSECTION DUA ARRAY
function intersectionLambat(arr1, arr2) {
    const hasil = [];

    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j] && !hasil.includes(arr1[i])) {
                hasil.push(arr1[i]);
            }
        }
    }

    return hasil;
}
function intersectionCepat(arr1, arr2) {
    const setArr2 = new Set(arr2);
    const hasil = [];

    for (const item of arr1) {
        if (setArr2.has(item) && !hasil.includes(item)) {
            hasil.push(item);
        }
    }

    return hasil;
}
```

tugas-1.js U

praktikum-5 > tugas > tugas-1.js > ...

```javascript
// FUNGSI B: GROUP ANAGRAM
function groupAnagram(words) {
    const groups = {};

    for (const kata of words) {
        const key = kata.split('').sort().join('');

        if (!groups[key]) {
            groups[key] = [];
        }

        groups[key].push(kata);
    }

    return Object.values(groups);
}

// FUNGSI C: ADAKAH a + b = c²
function cekKuadratLambat(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = 0; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (i !== j && j !== k && i !== k) {
                    if (arr[i] + arr[j] === arr[k] * arr[k]) {
                        return true;
                    }
                }
            }
        }
    }

    return false;
}

function cekKuadratCepat(arr) {
    const kuadratSet = new Set();

    for (const angka of arr) {
        kuadratSet.add(angka * angka);
```

tugas-1.js U

praktikum-5 > tugas > tugas-1.js > ...

```javascript
function cekKuadratCepat(arr) {
    }

    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (kuadratSet.has(arr[i] + arr[j])) {
                return true;
            }
        }
    }

    return false;
}

// DEMO FUNGSI A
console.log("=== Fungsi A: Intersection ===");

const arr1 = [1, 2, 3, 4, 5];
const arr2 = [3, 4, 5, 6, 7];

console.log("Lambat:", intersectionLambat(arr1, arr2));
console.log("Cepat :", intersectionCepat(arr1, arr2));

// DEMO FUNGSI B
console.log("\n=== Fungsi B: Group Anagram ===");

const words = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'];

console.log(groupAnagram(words));

// DEMO FUNGSI C
console.log("\n=== Fungsi C: a + b = c² ===");

const angka = [2, 7, 3, 4];

console.log("Lambat:", cekKuadratLambat(angka));
console.log("Cepat :", cekKuadratCepat(angka));
```

tugas-1.js U

praktikum-5 > tugas > tugas-1.js > ...

```javascript

// BENCHMARK DATA BESAR
console.log("\n=== Benchmark Data Besar ===");

const data1 = Array.from({ length: 3000 }, (_, i) => i);
const data2 = Array.from({ length: 3000 }, (_, i) => i + 1500);

ukurWaktu("Intersection Lambat O(n²)", () =>
    intersectionLambat(data1, data2)
);

ukurWaktu("Intersection Cepat O(n)", () =>
    intersectionCepat(data1, data2)
);

// Benchmark Fungsi C
const dataKuadrat = Array.from(
    { length: 300 },
    () => Math.floor(Math.random() * 100)
);

ukurWaktu("Kuadrat Lambat O(n³)", () =>
    cekKuadratLambat(dataKuadrat)
);

ukurWaktu("Kuadrat Cepat O(n²)", () =>
    cekKuadratCepat(dataKuadrat)
);
```

```
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5> cd tugas
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5\tugas> node tugas-1.js
=== Fungsi A: Intersection ===
Lambat: [ 3, 4, 5 ]
Cepat : [ 3, 4, 5 ]

=== Fungsi B: Group Anagram ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== Fungsi C: a + b = c² ===
Lambat: true
Cepat : true

=== Benchmark Data Besar ===
Intersection Lambat O(n²): 27 ms
Intersection Cepat O(n): 2 ms
Kuadrat Lambat O(n³): 0 ms
Kuadrat Cepat O(n²): 0 ms
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5\tugas>
```

Tugas 2 : Visualisasi Pertembuhan BIG O

JS tugas-2.js U

praktikum-5 > tugas > JS tugas-2.js > ...

```javascript
// TUGAS 2: Visualisasi Pertumbuhan Big O

function ukurWaktu(fn, n) {
    const mulai = Date.now();
    fn(n);
    return Date.now() - mulai;
}

function fn_O1(n) {
    return n + 1;
}

function fn_On(n) {
    let total = 0;

    for (let i = 0; i < n; i++) {
        total += i;
    }

    return total;
}

function fn_OnLogn(n) {
    let total = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            total += j;
        }
    }

    return total;
}
```

```javascript
}

function fn_On2(n) {
    let total = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            total += i + j;
        }
    }

    return total;
}

function benchmarkSemua(ukuranData) {
    console.log("=== Visualisasi Pertumbuhan Big O ===");

    for (const n of ukuranData) {
        console.log(`\nUkuran Data: ${n}`);

        console.log("O(1)     :", ukurWaktu(fn_O1, n), "ms");
        console.log("O(n)     :", ukurWaktu(fn_On, n), "ms");
        console.log("O(n log n):", ukurWaktu(fn_OnLogn, n), "ms");
        console.log("O(n²)    :", ukurWaktu(fn_On2, n), "ms");
    }
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

```
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5\tugas> node tugas-2.js
=== Visualisasi Pertumbuhan Big O ===

Ukuran Data: 100
O(1)     : 0 ms
O(n)     : 1 ms
O(n log n): 0 ms
O(n²)    : 1 ms

Ukuran Data: 500
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 1 ms
O(n²)    : 1 ms

Ukuran Data: 1000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n²)    : 2 ms

Ukuran Data: 5000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n²)    : 60 ms

Ukuran Data: 10000
O(1)     : 0 ms
O(n)     : 0 ms
O(n log n): 0 ms
O(n²)    : 223 ms
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-5\tugas>
```

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Modul 5 mengenai Analisis Kompleksitas Algoritma, dapat disimpulkan bahwa efisiensi algoritma merupakan faktor penting dalam pengembangan program, terutama ketika menangani data dalam jumlah besar. Melalui konsep Big O Notation, mahasiswa dapat memahami bagaimana waktu eksekusi dan penggunaan memori suatu algoritma bertumbuh seiring bertambahnya ukuran input.

Praktikum ini menunjukkan bahwa algoritma dengan kompleksitas rendah seperti O(1) dan O(n) jauh lebih efisien dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(2^n)$. Pengujian empiris menggunakan `Date.now()` membuktikan bahwa perbedaan performa menjadi semakin signifikan pada ukuran data besar. Selain itu, pemahaman tentang Space Complexity membantu dalam menilai trade-off antara kecepatan dan penggunaan memori.

Melalui latihan refactor, dapat dipahami bahwa pemilihan struktur data yang tepat seperti Set, Map, dan Object mampu meningkatkan performa secara drastis dengan mengurangi nested loop yang tidak efisien. Dengan demikian, praktikum ini memberikan pemahaman bahwa penulisan program tidak hanya berfokus pada kebenaran hasil, tetapi juga pada optimasi performa, skalabilitas, dan efisiensi sumber daya.

Secara keseluruhan, mahasiswa diharapkan mampu menganalisis, membandingkan, dan memperbaiki algoritma berdasarkan time complexity dan space complexity sehingga dapat menerapkan solusi pemrograman yang lebih optimal dalam berbagai permasalahan komputasi.